Header

El componente muestra la materia y el número de trabajo práctico dentro de un encabezado semántico. `App.js` lo incorpora con `<Header />`.

src/components/Header.js

```jsx
function Header() {
  return (
    <header className="encabezado">
      <div className="encabezado-contenido">
        <div>
          <p className="encabezado-materia">Lenguajes IV</p>
          <p className="encabezado-subtitulo">TP 06</p>
        </div>
      </div>
    </header>
  );
}

export default Header;
```

Componente Header: estructura y contenido.

src/App.css

```css
.encabezado {
  background-color: #ffffff;
  border-bottom: 1px solid #dfe8df;
}

.encabezado-contenido,
.pie-contenido {
  display: flex;
  align-items: center;
  width: min(100%, 960px);
  margin: 0 auto;
  padding: 18px 20px;
}
··· líneas intermedias omitidas ···
.encabezado-materia {
  font-size: 17px;
  font-weight: 700;
}

.encabezado-subtitulo {
  color: #536153;
  font-size: 13px;
}
```

Estilos del encabezado: contenedor, tipografía y borde inferior.

Footer

El pie de página está separado en su propio componente. Los estilos definen el borde superior, la alineación y el ajuste para pantallas pequeñas.

src/components/Footer.js

```
function Footer() {
  return (
    <footer className="pie">
      <div className="pie-contenido">
        <span>POR SIEMPRE MESSI</span>
      </div>
    </footer>
  );
}

export default Footer;
```

Componente Footer: estructura y texto.

src/App.css

```
.pie {
  background-color: #ffffff;
  border-top: 1px solid #dfe8df;
  color: #536153;
  font-size: 13px;
}

.pie-contenido {
  justify-content: space-between;
  gap: 12px;
··· líneas intermedias omitidas ···
  .pie-contenido {
    flex-direction: column;
    align-items: flex-start;
  }
```

Estilos del pie y adaptación a pantallas de hasta 540 px.

Creación del Blob

`crearBlob` construye un archivo de texto en memoria a partir de `TEXTO_ARCHIVO`, lo guarda en el estado y muestra su tipo y tamaño.

src/App.js

```
6  const TEXTO_ARCHIVO = 'Vamos Central NORTE.';
7
8  function App() {
9    const [blobDeTexto, setBlobDeTexto] = useState(null);
10   const [mensajes, setMensajes] = useState([]);
11
12   const crearBlob = () => {
13     const nuevoBlob = new Blob([TEXTO_ARCHIVO], { type: 'text/plain' });
14
15     setBlobDeTexto(nuevoBlob);
16     setMensajes([
17       'Estado: Blob creado correctamente.',
18       `Tipo: ${nuevoBlob.type}`,
19       `Tamaño: ${nuevoBlob.size} bytes`,
20     ]);
21   };
```

App.js: estado, texto de origen y creación del Blob.

> **Secuencia:** texto → `new Blob(...)` → estado de React → mensajes de resultado.

Descarga del Blob

Se comprueba que exista un Blob, se crea una URL temporal y se dispara la descarga mediante un enlace con el nombre `archivo.txt`. Después se libera la URL.

src/App.js

```
23  const descargarBlob = () => {
24    if (blobDeTexto === null) {
25      setMensajes(['Primero debe crear el Blob.']);
26      return;
27    }
28
29    const urlTemporal = URL.createObjectURL(blobDeTexto);
30    const enlace = document.createElement('a');
31
32    enlace.href = urlTemporal;
33    enlace.download = 'archivo.txt';
34    document.body.appendChild(enlace);
35    enlace.click();
36    enlace.remove();
37    URL.revokeObjectURL(urlTemporal);
38
39    setMensajes(['Archivo descargado correctamente.']);
40  };
```

App.js: URL temporal, enlace de descarga y liberación de recursos.

> **API usada:** `URL.createObjectURL` y `URL.revokeObjectURL`.

Recorte con slice()

La función obtiene los primeros 20 bytes del Blob, lee su contenido como texto y muestra el tamaño original, el tamaño de la porción y el resultado.

src/App.js

```
42  const usarSlice = async () => {
43    if (blobDeTexto === null) {
44      setMensajes(['Primero debe crear el Blob.']);
45      return;
46    }
47
48    const inicio = 0;
49    const fin = 20;
50    const porcionBlob = blobDeTexto.slice(inicio, fin, 'text/plain');
51    const contenidoPorcion = await porcionBlob.text();
52
53    setMensajes([
54      `Tamaño del Blob original: ${blobDeTexto.size} bytes`,
55      `Tamaño del Blob resultante: ${porcionBlob.size} bytes`,
56      `Contenido de la porción: "${contenidoPorcion}"`,
57    ]);
58  };
```

App.js: creación y lectura de una porción del Blob.

> **Detalle:** `slice(inicio, fin, tipo)` crea un nuevo Blob sin modificar el original.

Integración en la interfaz

El componente principal presenta el header, los tres botones que ejecutan las funciones del Blob, el área de mensajes y el footer.

src/App.js

```
60   return (
61     <div className="pagina">
62       <Header />
63
64       <main className="contenedor">
65         <h1>Ejemplo de uso de Blobs</h1>
66
67         <div className="botones">
68           <button type="button" onClick={crearBlob}>
69             Crear Blob de texto
70           </button>
71           <button type="button" onClick={descargarBlob}>
72             Descargar Blob
73           </button>
74           <button type="button" onClick={usarSlice}>
75             Usar slice()
76           </button>
77         </div>
78
79         <div className="resultado" aria-live="polite">
80           {mensajes.map((mensaje) => (
81             <p key={mensaje}>{mensaje}</p>
82           ))}
83         </div>
84       </main>
85
86       <Footer />
87     </div>
88   );
```

App.js: composición de componentes y acciones disponibles.